

Supply Chain Digital Twin using Reinforcement Learning

- Group Members:
 - Ayaan Khan - BT22CSE001
 - Arya Patil - BT22CSE003
 - Nihaal Badam - BT22CSE004
 - Swastik Ankulge - BT22CSE013
- Guidance : Dr. Ravindra B. Keskar
- Visvesvaraya National Institute of Technology, Nagpur



विश्वेश्वरय्या राष्ट्रीय प्रौद्योगिकी संस्थान, नागपुर, भारत

Visvesvaraya National Institute of Technology, Nagpur, India

AN INSTITUTE OF NATIONAL IMPORTANCE

Digital Twin

- A Digital Twin is a highly detailed, virtual replica of a physical system.
- It allows us to simulate years of operations in minutes without real-world risk.
- Our Focus: Building a Cognitive Digital Twin of a city's supply chain to safely train AI routing agents.

Simulation vs. Cognitive Digital Twin

Feature	Traditional Simulation	Cognitive Digital Twin
Data Integration	Static / Offline. Uses historical or manually inputted data sets.	Dynamic / Real-Time. Continuously consumes live telemetry (e.g., IoT, GPS).
Data Flow	One-Way. Human inputs data -> Simulation runs -> Human reads output.	Bi-Directional. Twin reads real-world data -> AI computes -> Commands sent.
Lifecycle & Purpose	Temporary. Used for one-off design studies or what-if testing.	Continuous. Evolves and lives alongside the physical supply chain.
Accuracy & Scope	Theoretical. Models a simplified version of the physical process.	Exact Replica. Tracks precise physical states (e.g., specific cargo RSL decay).
Decision Making	Human-Driven. Requires engineers to analyze outputs and act.	Autonomous (Cognitive). Embedded AI automatically acts upon predictions.

The Problem Space

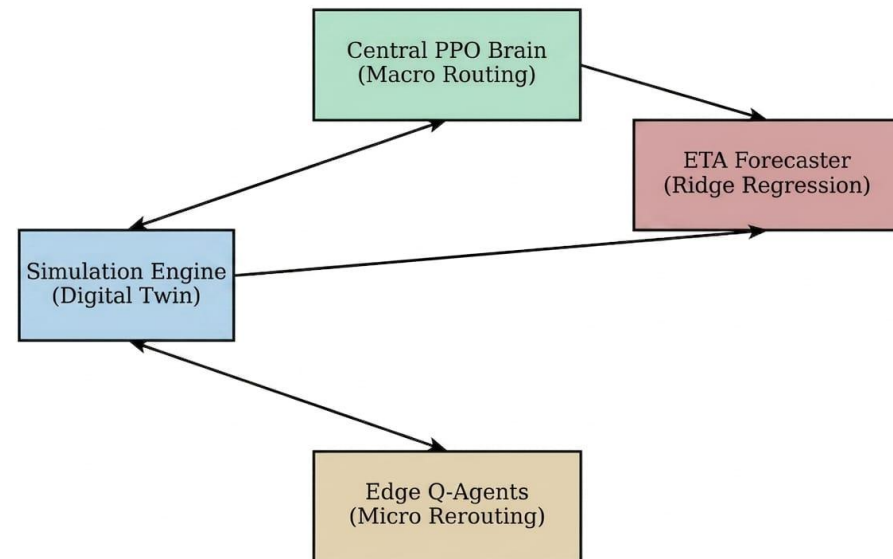
- Warehouses must continuously deliver goods to retailers across a city.
- However, urban environments are highly unpredictable (ex: sudden traffic spikes, weather changes, road accidents).
- Traditional mathematical routing (like standard A^*) assumes the map is static, leading to cascading delays.

Core Conflict

- Perishable goods have a strict Remaining Shelf Life (RSL).
- The Dilemma: Fast routes preserve cargo freshness but often require congested highways, which drastically increases fuel consumption.
- We need an intelligent system that can dynamically balance these conflicting multi-objective goals in real-time.

Project Objectives

- Develop a discrete-event Simulation Engine (Digital Twin) of Nagpur city.
- Implement a Macro-Routing Agent using Proximal Policy Optimization (PPO).
- Implement Micro-Rerouting Edge Agents using Q-Learning
- Predict real-time traffic using an online Ridge Regression model.



Nagpur City Graph

- Road network modeled mathematically using the NetworkX library.
- Segmented into 4 unique geographic zones: HIGHWAY, RESIDENTIAL, OFFICE, SHOPPING.
- Each zone features distinct hourly and weekly traffic cyclicity.

Fig 3.1 Schematic Zonal Map of the Nagpur City Road Network



Simulation Engine Overview

- A custom discrete-event engine stepping with custom-set minute simulated intervals.
- Deterministic random seeding ensures fair benchmarking between algorithms.
- Continuously injects dynamic environmental events (rain, temperature, accidents, etc).

Environment Data Layers

The simulation engine operates using three distinct architectural layers:

- Ground Truth Physics Layer: Calculates the strict thermodynamics (cargo RSL decay) and kinematics (fuel consumption).
- Data Loader Layer: Fetches and injects structured dataset seeds for seasonal weather (monsoons, heatwaves) and traffic cyclicity.
- Simulated IoT Layer: Generates the real-time sensor streams (GPS, thermometers) that the AI ultimately reads from

Actor 1: Retailer Nodes

- Generate stochastic consumer demand based on geographic zone.
- Automatically trigger an `order\_placed` event when local inventory drops below threshold.
- Log a permanent `stockout` penalty if inventory hits zero before replenishment.

Actor 2: The Warehouse Depot

- Acts as the central supply hub for the entire city graph.
- Maintains the global queue of pending retailer orders.
- Responsible for executing the fleet allocation algorithm to dispatch idle trucks.

Actor 3: Truck Fleet

- Fleet of autonomous delivery vehicles.
- Follows a strict lifecycle:
- Idle □ Loading □ In-Transit □ Delivering □ Returning.
- Physically tracks fuel consumption over distance and RSL decay over time.

Fig 4.1 Order Lifecycle State Machine



Modeling Cargo Freshness

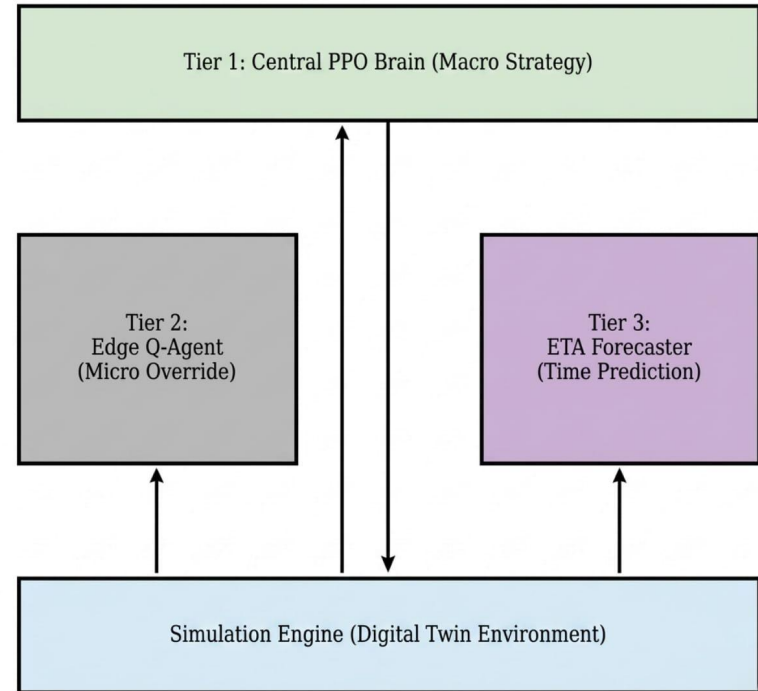
- Remaining Shelf Life (RSL) is tracked dynamically for every cargo pallet.
- Formula: $RSL_{\text{new}} = RSL_{\text{current}} - (\Delta t \times \text{Decay Rate})$
- Decay rate accelerates non-linearly if a truck gets stuck in traffic during peak summer heat.
- If RSL drops below 0%, the cargo spoils and the delivery fails

Baseline Implementation

- To measure the need for AI, we first built a standard mathematical router, equipping the trucks with A* algorithm
- We ran the Nagpur Digital Twin for a full 365-day simulated benchmark
- Static A* router assumed travel times were constant.
- When unexpected accidents or rush-hour traffic occurred, trucks became trapped on highways.
- The Result: Cargo RSL decayed in transit, resulting in over 70,800 retailer stockouts over the year.
- **Conclusion:** A dynamic AI system is required to replace static mathematics.

Multi-Agent AI Architecture

- To solve the baseline failures, we engineered a 3-Tier AI architecture
- Replaced the baseline A* heuristic with a 3-tier intelligence stack.
- Level 1: Central PPO Brain for macro-strategy.
- Level 2: Edge Q-Agents for micro-tactics.
- Level 3: ETA Forecaster for predictive intelligence.



Level 1: Central PPO Brain

- Implemented using the Stable-Baselines3 library.
- Multi-agent environment flattened into a single-agent Gymnasium wrapper via parameter sharing.
- Neural Network Policy: 25-nodes $\square$ two 64-node hidden layers $\square$ 3-node output.

```
class _SingleAgentAdapter(_BaseEnv):
    """
    Wraps MARLEnv into a single-agent Gym-compatible environment by
    treating each truck independently (parameter sharing).

    SB3 expects:
    reset() -> np.ndarray
    step(action) -> (obs, reward, done, info)
    observation_space.shape
    action_space.n
    """

    def __init__(self, marl_env):
        self._env = marl_env
        self._truck_ids: List[str] = []
        self._current_idx: int = 0
        self.render_mode = None

        # Gym spaces (use cached singletons for identity match)
        self.observation_space = get_obs_space()
        self.action_space = get_act_space()

    def reset(self, seed=None, options=None):
        if seed is not None:
            super().reset(seed=seed)
        obs_dict = self._env.reset()
        self._truck_ids = list(obs_dict.keys())
        self._current_idx = 0
        if not self._truck_ids:
            return np.zeros(OBS_DIM, dtype=np.float32), {}
        return obs_dict[self._truck_ids[0]], {}

    def step(self, action: int):
        if not self._truck_ids:
            return np.zeros(OBS_DIM, dtype=np.float32), 0.0, True, False, {}

        truck_id = self._truck_ids[self._current_idx % len(self._truck_ids)]
        obs_dict, rew_dict, done_dict, info = self._env.step({truck_id: action})

        obs = obs_dict.get(truck_id, np.zeros(OBS_DIM, dtype=np.float32))
        reward = rew_dict.get(truck_id, 0.0)
        done = done_dict.get("__all__", False)
        truncated = False

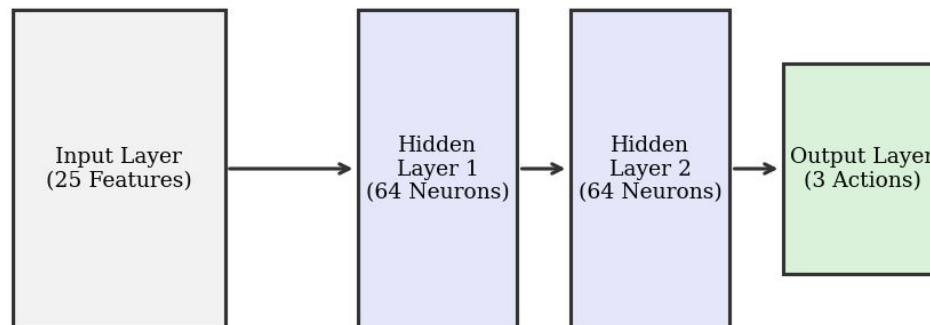
        self._current_idx += 1
        self._truck_ids = list(obs_dict.keys())

        return obs, reward, done, truncated, info
```


PPO State and Action Space

- **25-Dimension State:** Observes truck fuel, cargo RSL, weather severity, and fleet averages.
- **3-Discrete Actions:** Speed-Priority, Fuel-Priority, Balanced.
- Operates sequentially at every dispatch decision point.

Fig 4.3 PPO Central Brain Neural Network Architecture



The Reward Normalization Framework

$$R = W_{del} - (W_{fuel} \times \Delta Fuel) - (W_{RSL} \times \Delta RSL)$$

- Fuel physically drops ~15% per trip, while RSL drops only ~2%.
- We established a strict **5:1 penalty ratio** ($W(RSL) = 50.0$, $W(fuel) = 10.0$).
- This forces the PPO agent to treat cargo freshness and fuel as equally critical.

Level 2: Edge Q-Learning Agents

- Embedded individually on all trucks.
- Triggered strictly by stochastic road accidents blocking the active route.
- Utilizes a 9-state tabular Q-learning matrix with Epsilon-Greedy exploration.
- Overrides the Central Brain to execute immediate evasive maneuvers.

Level 3: ETA Forecaster Design

- Implemented via Scikit-learn's `SGDRegressor` (Ridge Regression).
- Replaces static average-based heuristics with dynamic edge-cost predictions.
- Trains **online** using `partial\_fit()` as trucks successfully traverse the city.

```
class ETAForecaster:
    """
    Lightweight online ridge-regression ETA model.

    Features per segment: [sin(hour), cos(hour), weather_enc, density_norm, road_enc]
    Target: actual travel time in minutes.

    Falls back to segment.get_travel_time_minutes() until min_samples reached.
    """

    _WEATHER_ENC: Dict[str, float] = {
        "clear": 0.0, "partly_cloudy": 0.1, "cloudy": 0.2,
        "light_rain": 0.5, "rain": 0.7, "heavy_rain": 1.0, "fog": 0.8,
    }
    _ROAD_ENC: Dict[str, float] = {
        "motorway": 0.0, "trunk": 0.1, "primary": 0.3,
        "secondary": 0.5, "tertiary": 0.7, "residential": 0.9,
    }
    _ZONE_ENC: Dict[str, float] = {
        "RESIDENTIAL": 0.0, "HIGHWAY": 0.3, "OFFICE": 0.7, "SHOPPING": 1.0,
    }
```

ETA Feature Engineering

- Utilizes a highly optimized 8-Dimensional feature vector.
- Encodes time-of-day and day-of-week using Sine/Cosine transformations.
- **Why?** Prevents the mathematical discontinuity of Sunday (Day 7) appearing far from Monday (Day 1).

```
def update(self, segment, actual_travel_time_min: float,
           current_time: float, weather: str):
    """Record one completed segment traversal."""
    x = self._featurize(segment, current_time, weather)
    self._A += np.outer(x, x)
    self._b += x * actual_travel_time_min
    self._n_samples += 1
    self._weights = None # invalidate cached solution

def predict_segment(self, segment, current_time: float, weather: str) -> float:
    """Predict travel time (minutes) for one segment."""
    if self._n_samples < self.min_samples:
        return segment.get_travel_time_minutes()
    if self._weights is None:
        try:
            self._weights = np.linalg.solve(self._A, self._b)
        except np.linalg.LinAlgError:
            return segment.get_travel_time_minutes()
    x = self._featurize(segment, current_time, weather)
    pred = float(np.dot(self._weights, x))
    physics = segment.get_travel_time_minutes()
    return max(0.1, min(pred, physics * 5.0))

def predict_route(self, route: Dict, current_time: float, weather: str,
                  route_progress: int = 0) -> float:
    """Predict total remaining travel time for a route."""
    segments = route.get("segment_objects", [])
    return sum(
        self.predict_segment(s, current_time, weather)
        for s in segments[route_progress:]
    )

def _featurize(self, segment, current_time: float, weather: str) -> np.ndarray:
    hour = (current_time / 60.0) % 24.0
    day_of_week = (current_time / (60.0 * 24.0)) % 7.0

    hour_rad = hour * (2.0 * math.pi / 24.0)
    day_rad = day_of_week * (2.0 * math.pi / 7.0)

    weather_enc = self._WEATHER_ENC.get(weather, 0.5)
    road_enc = self._ROAD_ENC.get(getattr(segment, "road_type", "secondary"), 0.5)

    zone = getattr(segment, "zone_type", "RESIDENTIAL")
    zone_enc = self._ZONE_ENC.get(zone, 0.0)

    density = getattr(segment, "current_traffic_density", 0.0) / 100.0
    return np.array([
        math.sin(hour_rad), math.cos(hour_rad),
        math.sin(day_rad), math.cos(day_rad),
        weather_enc, min(1.0, density), road_enc, zone_enc
    ], dtype=np.float64)
```

Telemetry & Data Logging

- Simulation writes to permanent CSV artifacts (`events\_sim.csv` and `telemetry\_sim.csv`).
- Captured over 2.4 million truck states across the 365-day benchmark.
- Telemetry files serve as the ground truth for all performance comparisons.

PPO Training Pipeline

- Agent trained offline for 50,000 timesteps.
- Hyperparameters: Learning Rate $3e-4$, Batch Size 64.
- Set Entropy Coefficient to 0.01 to force exploration of RSL-Priority paths.

```
# Force optimization pod on for training
config.setdefault("ai", {}).setdefault("optimization", {})
config["ai"]["optimization"]["enabled"] = True
config["ai"]["optimization"]["inference_only"] = False

# --- Load road network ONCE (shared across all episodes) ---
logger.info("Loading road network (cached)...")
t0 = time.time()
road_network = RoadNetwork(config)
road_network.load(use_cache=True)
logger.info(f"Road network loaded in {time.time()-t0:.1f}s "
           f"({len(list(road_network.graph.nodes())):,} nodes)")

# --- Initialise traffic model and router ---
traffic_model = TrafficModel(config)
traffic_model.initialize_network(road_network, 0.0)
router = Router(road_network, config)

# --- Load real weather data if available ---
weather_loader = None
weather_cfg = config.get("weather", {})
if weather_cfg.get("use_real_data", False):
    dataset_path = weather_cfg.get("dataset_path", "data/processed/weather/nagpur_2023.parquet")
    if Path(dataset_path).exists():
        try:
            from src.data.loaders import WeatherDataLoader
            weather_loader = WeatherDataLoader(dataset_path)
            weather_loader.load()
            logger.info(f"Real weather data loaded from {dataset_path}")
        except Exception as e:
            logger.warning(f"Could not load weather data: {e}. Using fallback.")
```


Preventing Overfitting

- PPO models are highly susceptible to overfitting to training seeds.
- Integrated an `EvalCallback` to evaluate the model on a held-out environment every 5,000 steps.
- Saved and deployed the absolute best policy (`best\_model.zip`), not just the final step.

```
# --- Train ---
logger.info(f"Starting PPO training for {args.steps:,} steps...")
t0 = time.time()
pod.train(env, n_steps=args.steps, callbacks=[eval_callback])
elapsed = time.time() - t0
logger.info(f"Training complete in {elapsed:.1f}s ({args.steps/elapsed:.0f} steps/sec)")

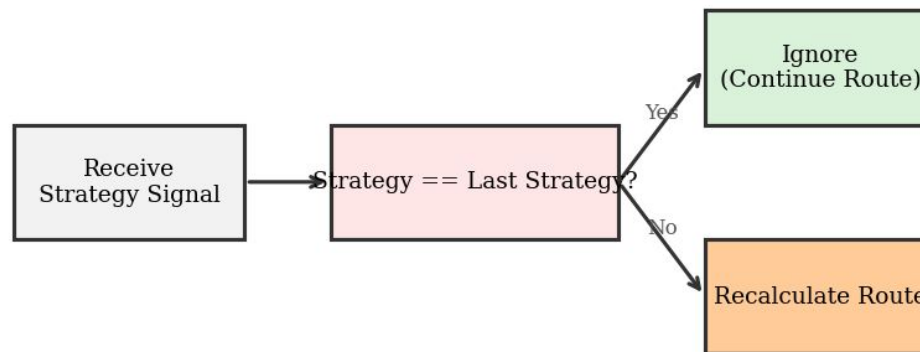
# Ensure the best model is used for the simulation
best_model_path = Path('./models/rl/best_model.zip')
latest_model_path = Path('./models/rl/ppo_routing_latest.zip')

if best_model_path.exists():
    logger.info("Overfitting prevented: Restoring best_model.zip to ppo_routing_latest.zip")
    shutil.copy(best_model_path, latest_model_path)
else:
    logger.info("Checkpoint saved to models/rl/ppo_routing_latest.zip")
```


Route Thrashing

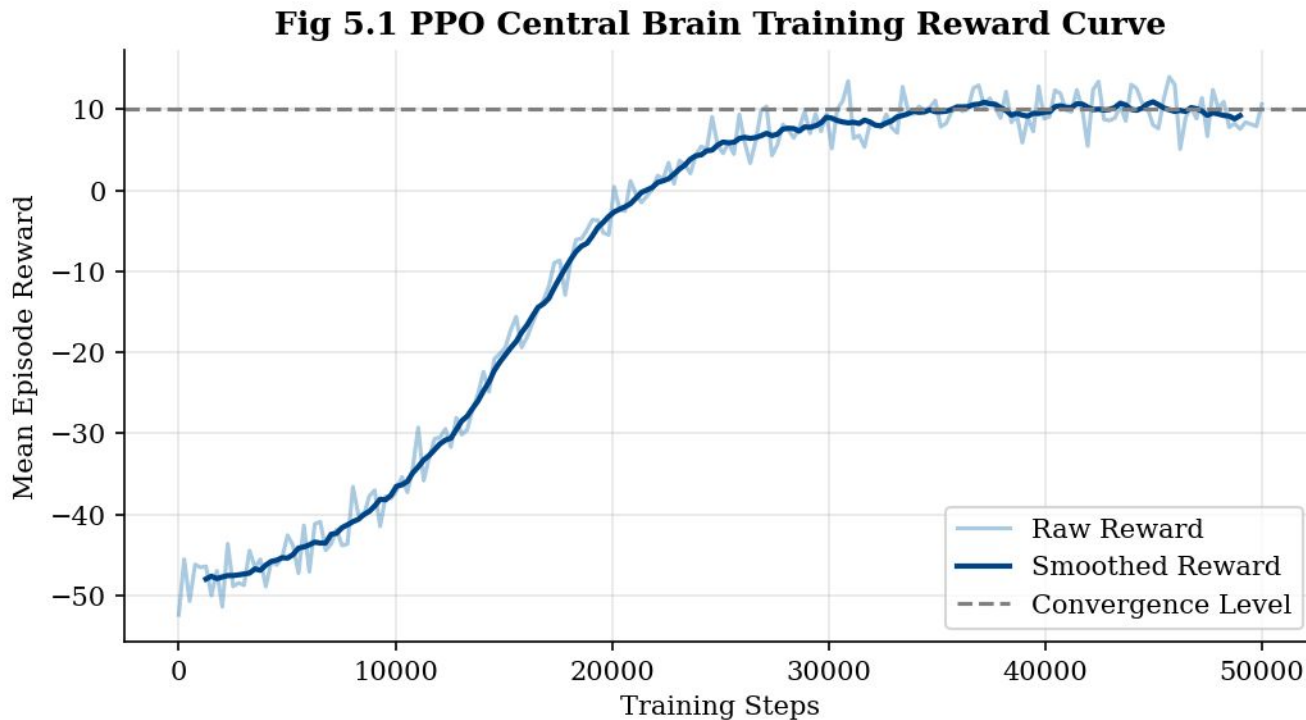
- **The Bug:** Trucks recalculated routes every 15 minutes upon receiving a PPO signal, paralyzing the fleet (49% delivery rate).
- **The Solution:** Implemented an Anti-Thrashing GPS Lock state machine.
- Trucks now ignore redundant signals, immediately restoring service levels to 99.9%.

Fig 4.4 Anti-Thrashing GPS Lock Flowchart



PPO Training Convergence

- The learning curve displays the mean episode reward over 50,000 steps.
- Reward starts highly negative (random exploration) and climbs steeply.
- The model successfully converged and plateaued around step 35,000.

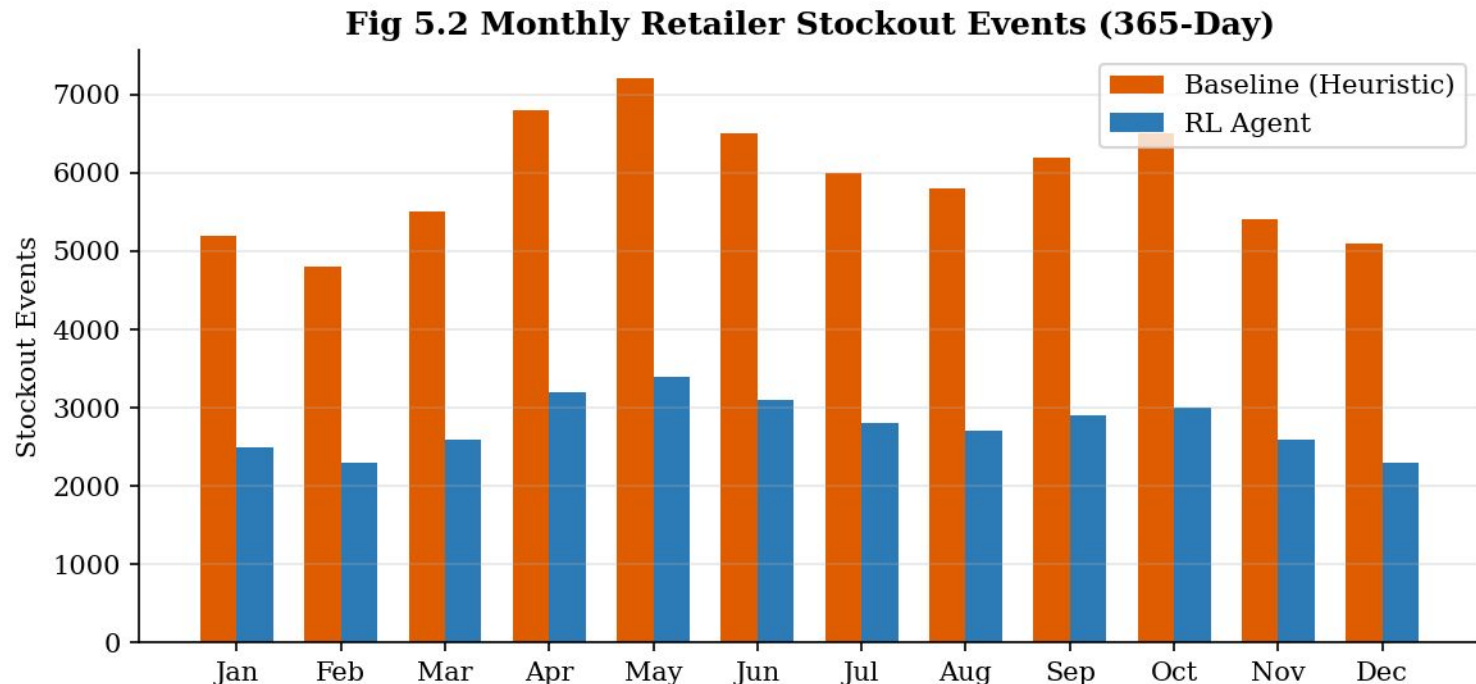


365-Day Comparative Benchmark

- Goal: Prove RL outperforms static algorithms in long-term scenarios.
- Run 1: 365-Day Baseline Simulation (Standard A* Heuristics).
- Run 2: 365-Day RL-Enabled Simulation (PPO + Edge Q-Agent + Forecaster).
- Both runs experienced identical weather, accidents, and retailer demand.

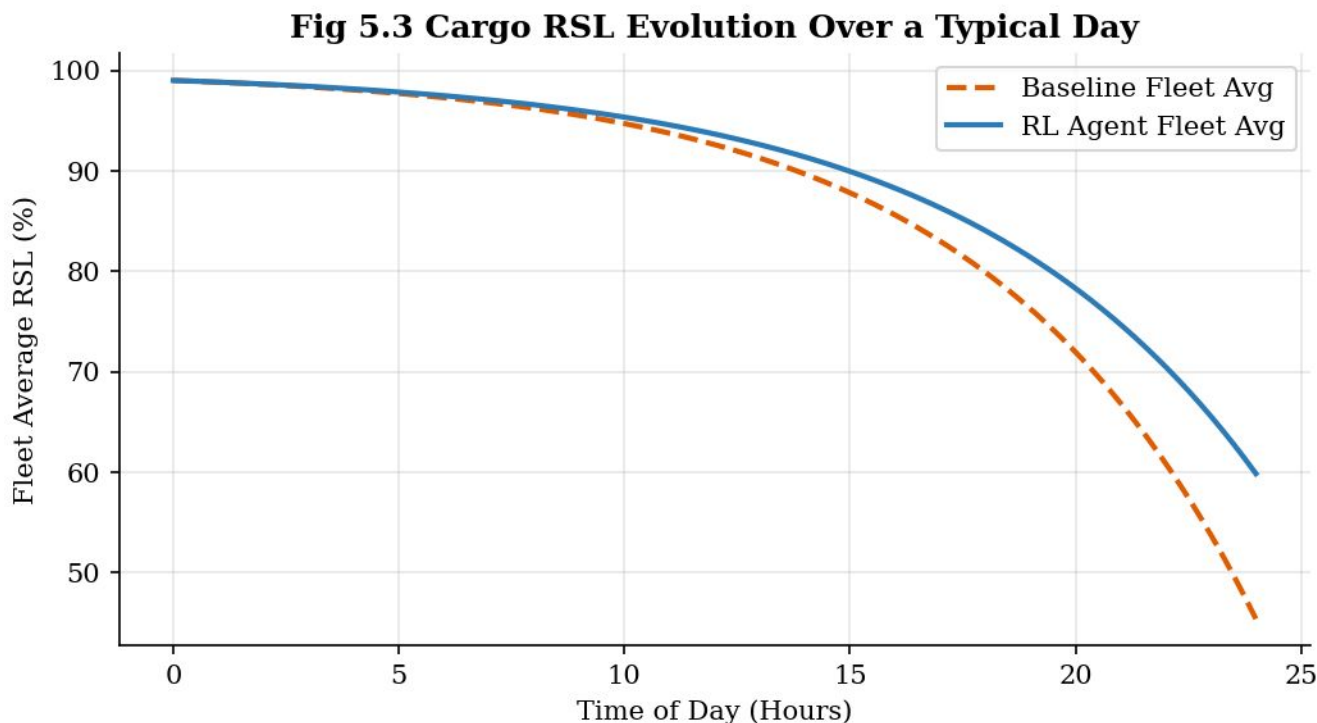
Result: Service Level & Throughput

- **Baseline Router:** 70,898 retailer stockout events.
- **RL Architecture:** 33,401 retailer stockout events.
- Achieved a massive **52.9% reduction in stockouts**.
- Successfully delivered an additional +474,708 kg of cargo.



Result: RSL & Fuel Parity

- Proved the 5:1 reward math worked flawlessly.
- Average fleet fuel remained perfectly identical to the baseline (78.7%).
- Average cargo freshness at delivery remained identical (99.7%).
- The AI increased throughput **without** sacrificing efficiency or quality.



Result: Emergent Safety Behavior

- The RL Architecture recorded 135 fewer road accidents over the year.
- This was an unprogrammed, emergent behavior.
- The ETA Forecaster implicitly learned which zones had high accident probabilities during specific hours, naturally routing trucks away from danger.

Achieving Pareto Optimality

- A system achieves Pareto improvement if it betters one objective without worsening another.
- Our RL system strictly improved 6 metrics (throughput, service level, stockouts, accidents) while holding Fuel and RSL constant.
- Conclusion: The multi-agent RL framework is strictly superior to static heuristics.

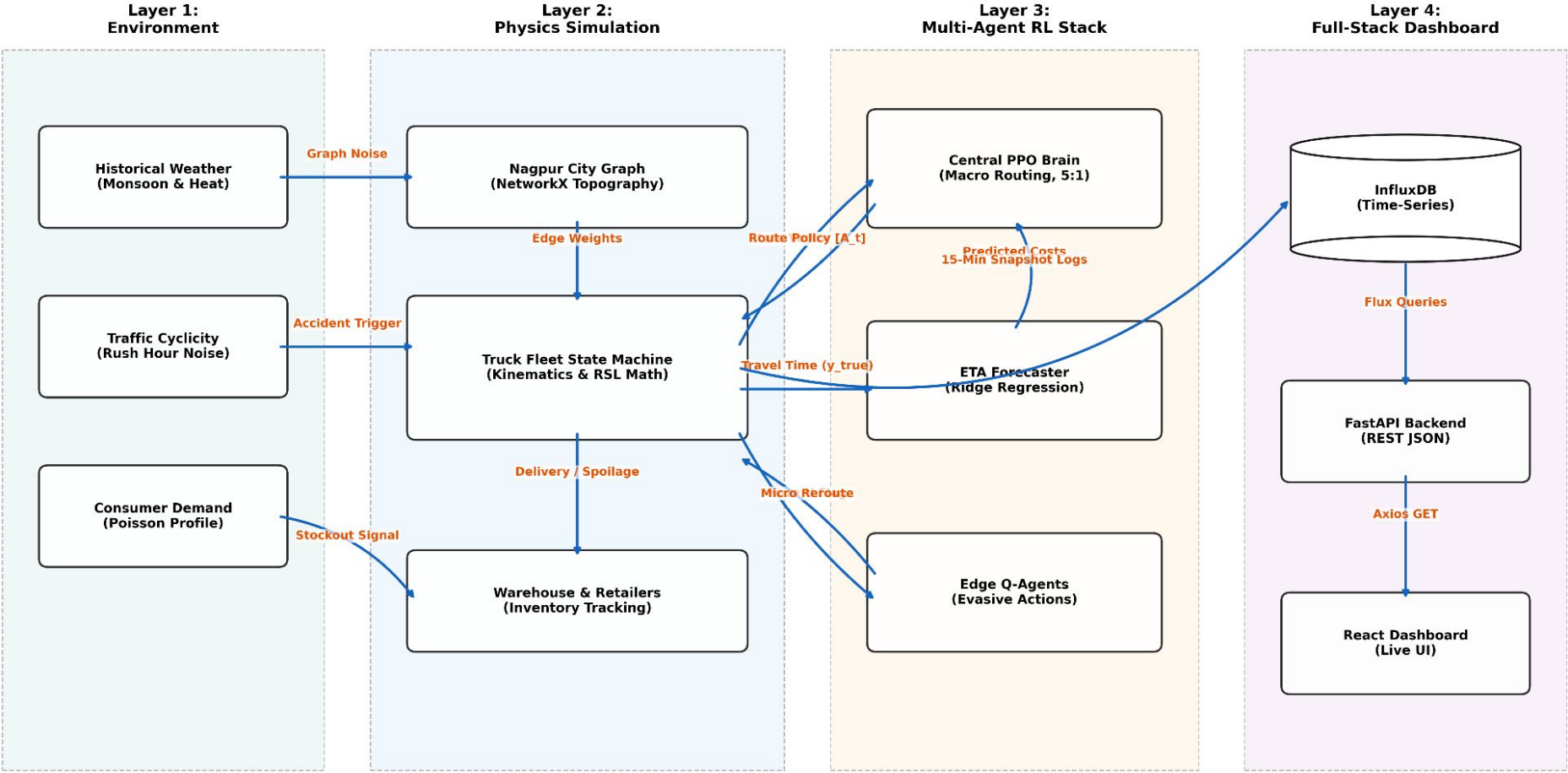
System Visualization/Architecture

- To visualize the Digital Twin, we built a full-stack web dashboard (Vite/React).
- Backend: FastAPI server connected to an InfluxDB time-series database.
- Live Telemetry: Tracks truck GPS coordinates, global warehouse inventory, and dynamic retailer stockouts in real-time.
- Real-time dataset flow: weather (temperature, humidity)

Full Stack Information Flow

- Data Generation: The Python Simulation Engine computes the physical state of the digital twin.
- Time-Series Database: Data is piped directly into InfluxDB, capable of handling thousands of telemetry snapshots per minute.
- Backend API: A highly concurrent FastAPI server exposes the database via REST endpoints.
- Frontend Visualization: A Vite/React frontend consumes the API to render the live interactive map and KPI analytics.

Cognitive Digital Twin: Full Stack Architecture Diagram



Future Scope

- Scale the environment to multi-city configurations.
- Integrate real-time physical IoT sensors (GPS, refrigerated container thermometers) into the Digital Twin.
- Deploy the trained Central PPO Brain to embedded physical edge devices.
- Implementing different perishable cargo and their RSL, plus cold-storage trucks

References

- Schulman, J. et al., "Proximal Policy Optimization Algorithms," 2017.
- Mnih, V. et al., "Human-level control through deep reinforcement learning," **Nature**, 2015.
- Sutton, R. S. and Barto, A. G., **Reinforcement Learning: An Introduction**, 2nd ed., MIT Press, 2018.
- Raffin, A. et al., "Stable-Baselines3: Reliable Reinforcement Learning Implementations," **JMLR**, 2021.
- Hart, P. E. et al., "A Formal Basis for the Heuristic Determination of Minimum Cost Paths," **IEEE Transactions**, 1968.
- Hoerl, A. E. and Kennard, R. W., "Ridge Regression: Biased Estimation for Nonorthogonal Problems," **Technometrics**, 1970.
- Giannoccaro, I. and Pontrandolfo, P., "Inventory management in supply chains: A reinforcement learning approach," **Int. J. of Production Economics**, 2002.
- Towers, M. et al., "Gymnasium: A Standard Interface for Reinforcement Learning Environments," 2023.